SCOUT REMOTE

Mobile Companion for Microsoft Scout

Product & Technical Design Specification

Design intent

A secure mobile control plane for checking Scout status, reviewing bridge-owned chats and activity, sending bounded instructions, viewing artifacts, and understanding usage - without depending on undocumented Scout internals.

Version 1.2

14 August 2026

Status: Proposed reference architecture

Important product boundary

Scout Remote remains a companion to the documented Windows/macOS Scout desktop application. The design does not assume a public API for native Scout chat/session control. The stable contract is the app-owned bridge protocol, delivered through the Agent Plugin package and local MCP bridge/helper.

Revision 1.2 - Android-first release

Launch sequencing is now explicit: Phase 0 and Phase 1 target Android first, with Google Play packaging and Android-specific validation. iOS remains a planned Phase 4 target on the same shared domain, transport, identity, and platform-service abstractions; iOS-specific Entra configuration, Keychain storage, background behavior, and App Store work are deferred until that phase.

1. Executive summary

Scout Remote is a companion experience, not a replacement for the Scout desktop client. The mobile app provides visibility and bounded remote interaction while preserving Scout's desktop approval and safety model. The first public

Recommended architecture

Use one Scout Remote Microsoft Entra public-client app registration across mobile and desktop bridge clients. Mobile and desktop authenticate independently with MSAL and keep separate secure token caches. Microsoft Graph + the OneDrive App Folder is the durable cross-device data plane. Package the desktop bridge as an Agent Plugins 1.0 bundle containing a Scout skill and a local MCP bridge/helper; the helper owns Graph authentication and protocol I/O.

• **Launch strategy:** Android first for the spike and MVP; iOS is a planned follow-on release using the same shared architecture and contracts.
• **Primary use case:** check whether Scout is active, what it is working on, what it recently completed, and whether it needs desktop attention.
• **Secondary use case:** send a bounded instruction from mobile and receive a structured asynchronous response.
• **Desktop bridge identity:** same Entra app ID and signed-in M365 user as mobile, but a separate desktop MSAL session/token cache; never transfer mobile tokens.
• **Plugin packaging:** Agent Plugins 1.0 is the distribution format because it can bundle Agent Skills and MCP servers. Until Scout documents native plugin installation, install the contained skill and MCP server through Scout-supported mechanisms.
• **Security posture:** delegated identity, least privilege, OS-protected token caches, schema validation, idempotent commands, expiration, and no remote bypass of Scout approvals.

Area	MVP decision	Reason
Mobile release	Android first; iOS planned	Reduces launch surface while preserving a cross-platform design.
Mobile framework	.NET MAUI reference implementation; React Native acceptable	Keep shared domain/transport code and platform-service abstractions from day one.
Identity	One Entra public-client app registration; Android + desktop configured first	Add iOS platform configuration when the iOS phase begins; no token sharing.
Data plane	Graph + OneDrive App Folder	Cross-device, user-owned, least-privilege storage; validate tenant support at build time.
Desktop bridge	Agent Plugins 1.0 bundle: SKILL.md + local MCP bridge	Portable packaging; MCP helper provides deterministic Graph identity and protocol I/O.
Chat access	Bridge-owned conversations	Avoids dependency on undocumented native chat schema/API.

2. Goals, non-goals, and product principles

2.1 Goals

• Provide a trustworthy “what is Scout doing?” mobile companion, launching first on Android with iOS following on the same architecture.
• Expose recent activity, task state, remote conversation history, artifacts, and automation outcomes in a compact mobile UX.
• Allow bounded mobile-to-Scout requests without requiring inbound networking to the user's PC/Mac.
• Keep the data model durable, versioned, exportable, and independent of a specific mobile framework or platform release sequence.
• Preserve shared domain, transport, identity, secure-storage, background-work, and notification abstractions so iOS is an implementation/testing phase rather than an architectural rewrite.
• Preserve the user's ability to keep operating when Scout or the desktop is temporarily offline.

2.2 Non-goals

• Do not reproduce the full Scout desktop UX on a phone.
• Do not make iOS a launch-blocking requirement for the Android MVP.
• Do not bypass shell/browser/file approval policies or remotely approve sensitive actions unless Microsoft exposes an explicit supported approval API in the future.
• Do not parse or mutate undocumented Scout databases, caches, or session files as the production contract.
• Do not place API keys, refresh tokens, device tokens, or D365 credentials into OneDrive protocol JSON.

2.3 Product principles

Principle	Design consequence
Explicit over magical	Every remote command has an ID, timestamp, expiry, status, and auditable result.
Asynchronous first	The app remains useful even if Scout is offline for hours.
Android first, architecture neutral	Ship one mobile target first without coupling domain/transport contracts to Android-only APIs.
Platform services behind interfaces	Secure storage, background work, notifications, auth callbacks, and external-app launching are abstracted for later iOS implementation.
Least privilege	Default to App Folder access; request broader scopes only for an explicit feature.
Desktop safety boundary	Mobile can request; desktop Scout decides/asks for approvals under its own policy.
Stable bridge contract	App and Scout bridge communicate through versioned JSON, not internal Scout state.

3. Current Microsoft platform assumptions

The design is grounded in Microsoft documentation available as of 14 August 2026. Scout is a Frontier product and Agent Plugins 1.0 is newly published, so production implementation should re-validate these assumptions at build time.

Assumption	Design impact
Scout is currently desktop-only on Windows 11+ and macOS 12+.	Mobile is a companion/remote surface, not a native Scout runtime.
Scout supports heartbeat and scheduled/condition-triggered automations.	Bridge processing can occur without an interactive user turn.
Scout supports custom SKILL.md skills. Scout admin policy also exposes MCP-server controls.	Use a skill for orchestration and a local MCP server/helper for deterministic bridge operations.
Agent Plugins 1.0 packages Agent Skills and MCP servers in one portable directory.	Distribute Scout Remote Bridge as a plugin package; keep components installable separately.
Microsoft Graph exposes OneDrive App Folder access via Files.ReadWrite.AppFolder.	Both mobile and desktop bridge use the same Entra application ID and signed-in user to reach the same approot.
MSAL supports native public-client authentication on mobile and desktop.	No client secret; each device/client maintains its own secure token cache.

Unsupported assumptions to avoid

Do not assume Scout currently installs Agent Plugins 1.0 packages natively. Public Scout docs document custom skills and expose MCP-server administration, but do not yet document an Agent Plugins package installer. For v1, treat Agent Plugins as the portable source/distribution layout, then install/register the contained skill and MCP server by the supported Scout mechanisms. Also do not assume a public Scout endpoint for native conversations, approvals, arbitrary task cancellation, or event subscription.

4. Logical architecture

The architecture uses OneDrive as a durable mailbox/state store. The mobile app and Scout never need a direct network connection. The Android client and desktop bridge authenticate independently but share the same Scout Remote Entra application ID. The later iOS client plugs into the same app-facing contracts and data plane.

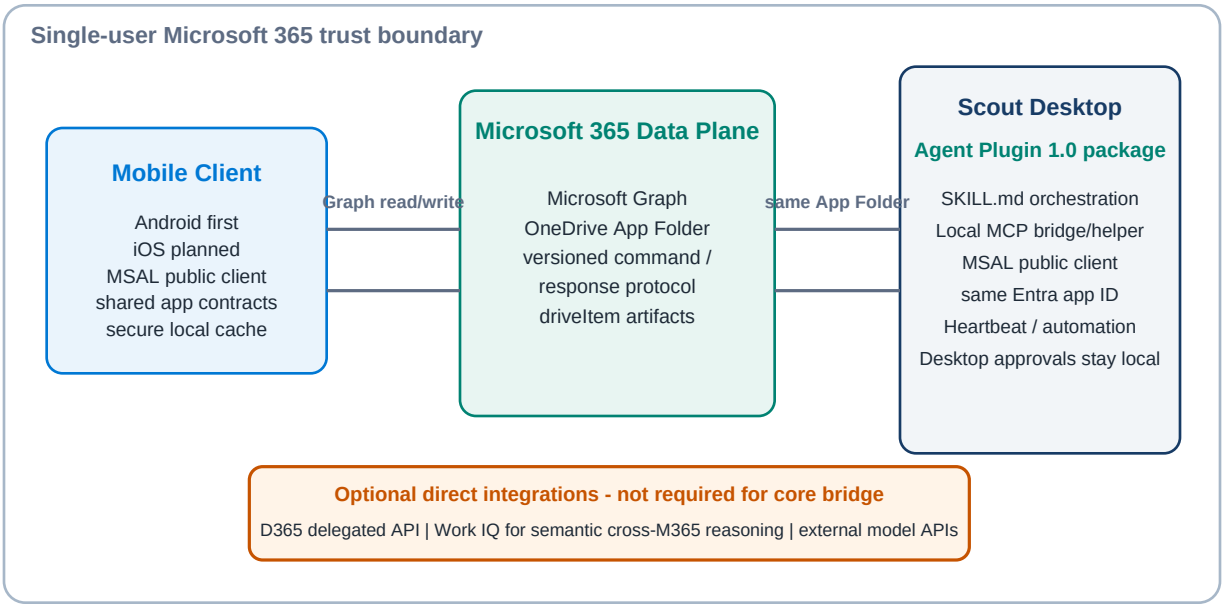


Figure 1. Recommended logical architecture

Component	Responsibilities
Scout Remote mobile app	Android is the first release target. Shared domain/transport/application code remains cross-platform; iOS later supplies platform-specific services behind the same interfaces.
Microsoft Graph / OneDrive	Durable protocol storage and sync boundary. App Folder is addressed by the shared Entra application ID.
Scout Remote Agent Plugin	Portable package containing the Scout skill plus local MCP bridge/helper. Plugin packaging is not assumed to be natively installable by Scout.
Local MCP bridge/helper	Desktop MSAL sign-in; Graph App Folder access; protocol reads/writes; deterministic bridge operations. Keeps its own token cache.
Scout skill + heartbeat/automation	Agent-facing orchestration and scheduled polling. Uses the MCP bridge for data-plane operations; Scout approvals remain local.

5. End-to-end command flow

A remote message is a durable job. The user can close the app immediately after sending; processing resumes when Scout is available.

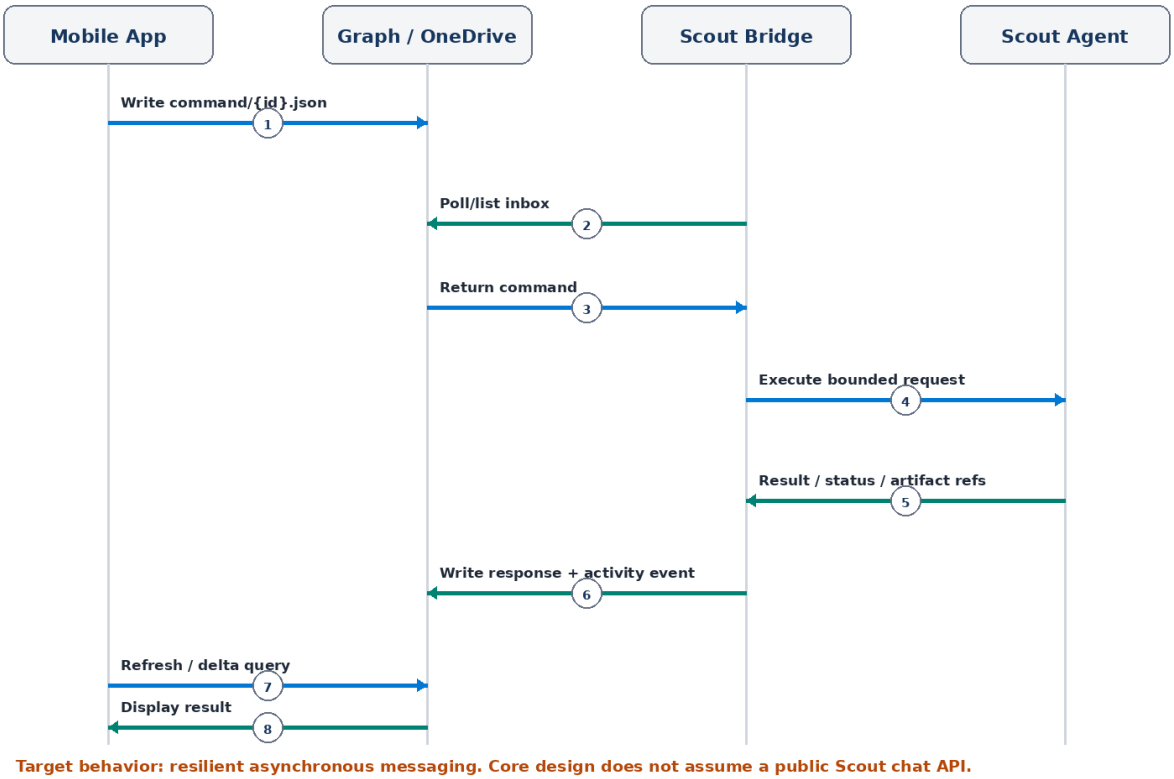


Figure 2. Mobile-to-Scout asynchronous command sequence

5.1 Processing semantics

- At-least-once observation, exactly-once effect where feasible: Scout may observe the same command more than once, so command IDs and processed receipts make execution idempotent.
- Commands expire. A stale instruction must not suddenly run hours or days later unless the command explicitly permits delayed execution.
- The bridge writes an acknowledgement quickly, then a final response later. Long-running work should emit progress events.
- If a request would require a sensitive Scout action, the response should switch to waiting_for_desktop_approval and explain what the user must do on the desktop.

6. OneDrive protocol and storage layout

Use the OneDrive App Folder as the default location because it provides a stable app-specific namespace and a least-privilege Graph permission.

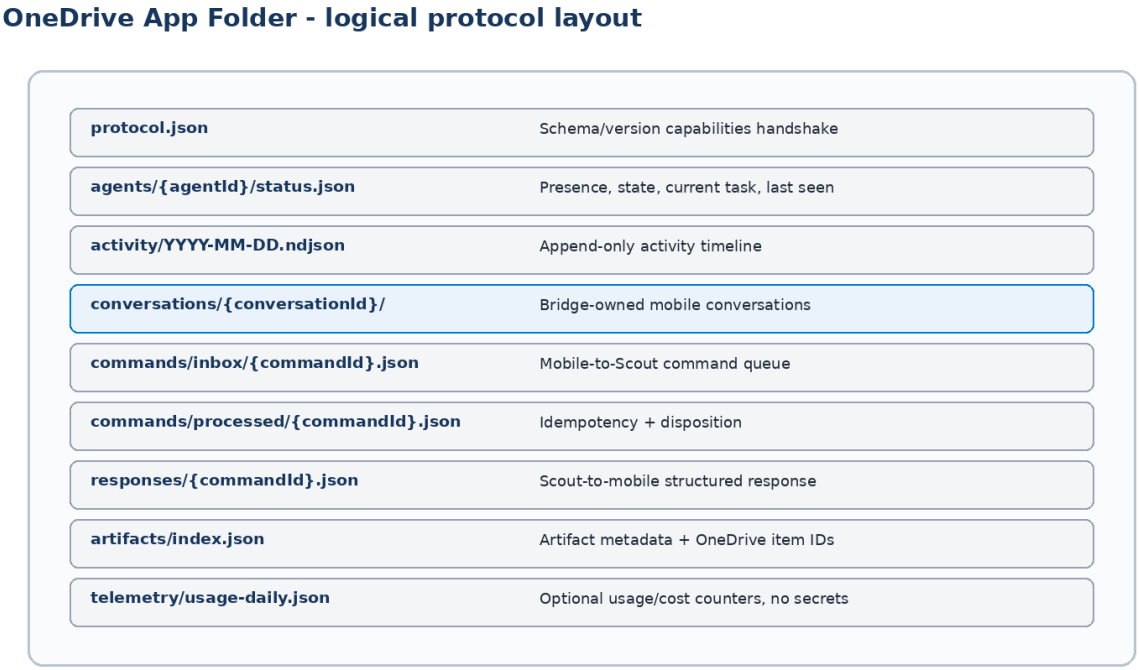


Figure 3. Versioned bridge protocol layout

6.1 Protocol handshake

protocol.json

```
{
  "protocol": "scout-remote",
  "version": "1.0",
  "agentId": "primary",
  "bridgeVersion": "1.0.0",
  "capabilities": [
    "status.read", "activity.read", "message.send",
    "summary.request", "artifacts.read"
  ],
  "nativeScoutChatMirror": false,
  "remoteApprovals": false
}
```

6.2 Agent status contract

agents/primary/status.json

```
{
  "schemaVersion": 1,
  "agentId": "primary",
  "state": "working",
  "lastSeenUtc": "2026-08-13T22:32:10Z",
  "currentTask": {
    "title": "Reviewing architecture notes",
    "startedUtc": "2026-08-13T22:25:00Z",
    "progressText": "3 of 5 sources reviewed"
  },
  "attention": {
    "required": false,
    "reason": null
  }
}
```

7. Command and response contracts

7.1 Allowed MVP command types

Command	Purpose	Desktop risk
message	Send a bounded natural-language instruction to the bridge-owned remote conversation.	Low to variable; bridge must enforce policy.
request_status	Ask Scout to publish a fresh status snapshot.	Low.
request_summary	Ask for a concise summary of recent bridge activity/work.	Low.
request_artifact	Ask Scout to publish or link an artifact it created.	Low to variable.
cancel_request	Request cancellation of bridge-owned work where cancellation is actually supported.	Best effort only; never claim success without confirmation.

commands/inbox/{commandId}.json

```
{
  "schemaVersion": 1,
  "commandId": "01J5R9Q8C5R0E6Y2M4P7K2P9AP",
  "conversationId": "remote-2026-08-13",
  "type": "message",
  "createdUtc": "2026-08-13T22:40:00Z",
  "expiresUtc": "2026-08-14T02:40:00Z",
  "requestedBy": { "entraObjectId": "<oid>" },
  "text": "Give me a concise status update and link the latest artifact.",
  "constraints": {
    "maxWorkIqCalls": 1,
    "allowSensitiveActions": false
  }
}
```

responses/{commandId}.json

```
{
  "schemaVersion": 1,
  "commandId": "01J5R9Q8C5R0E6Y2M4P7K2P9AP",
  "status": "completed",
  "startedUtc": "2026-08-13T22:45:01Z",
  "completedUtc": "2026-08-13T22:46:14Z",
  "message": "Architecture review is complete. Two follow-ups remain.",
  "artifacts": [
    { "name": "architecture-review.docx", "driveItemId": "<id>" }
  ],
  "usage": { "workIqCalls": 0, "modelTokens": null }
}
```

8. Conversation model

The app must distinguish between a stable remote conversation it owns and Microsoft Scout's native desktop conversation history.

8.1 Bridge-owned remote conversations - supported baseline

- Each mobile thread has a conversationId and an append-only event stream.
- Mobile messages become commands; Scout responses become response events. The entire remote thread is therefore reproducible from protocol data.
- A remote thread can include links to Scout-created artifacts, activity events, and "desktop attention required" markers.
- This model works even if Microsoft changes the internal storage format of Scout native sessions.

8.2 Native Scout chat mirroring - optional adapter

> **Do not make native chat mirroring an MVP dependency**
>
> Microsoft documentation confirms Scout can remember and search past session history, but this specification does not assume a public, versioned API for enumerating or posting into those sessions. Treat any native-chat export as a capability adapter with a feature flag.

- Preferred future path: use an official Scout API if Microsoft publishes one.
- Acceptable interim path: an explicit Scout command/skill that publishes selected conversation summaries or exports to the app folder.
- Avoid: scraping the Scout UI, reading undocumented caches/databases, or mutating internal session files.

9. Mobile information architecture and UX

Screen	Primary content	Primary action
Home	Agent state, last seen, current task, pending attention, latest result.	Ask for status / open active thread.
Activity	Chronological events grouped by day and command.	Filter by work, artifact, error, Work IQ.
Chats	Bridge-owned remote threads; optional mirrored native summaries.	Send message / request summary.
Artifacts	Recent files, type, size, created by command, OneDrive link.	Open/share in Microsoft 365 app.
Automations	Published automation names, last run/result, next known run when available.	View details; editing is future/feature-flagged.
Usage	Work IQ call counts, model usage when provided, request-level cost notes.	Set cost guardrails.
Settings	Account, agent binding, sync cadence, privacy, diagnostics.	Re-authenticate / reset bridge.

9.1 Home screen states

State	Presentation	Meaning
Online / idle	Green dot, last seen < configured threshold	Bridge recently published status; no active task.
Working	Blue progress treatment + current task	Scout reports active bridge-owned work.
Waiting for desktop	Amber attention card	A sensitive action or local-only step needs the desktop.
Offline / stale	Gray indicator + explicit last seen	No recent bridge update; commands remain queued.
Error	Red card with retry-safe diagnostic	Bridge or protocol error requires attention.

9.2 Notification policy

- Notify when a user-submitted command completes, fails, or enters waiting_for_desktop_approval.
- Do not notify for every heartbeat. Collapse routine activity into a digest.
- If background push infrastructure is not deployed, use local refresh/background fetch as available and label data freshness explicitly.

10. Identity, authorization, and token handling

10.1 Mobile authentication

• Register Scout Remote as a public client application in Microsoft Entra ID.
• **Android MVP:** configure Android plus the required desktop/native redirect platforms first. Use MSAL authorization code flow with PKCE for Android sign-in.
• Do not ship a client secret in the mobile binary.
• Store the Android token cache through Keystore-backed secure storage behind a platform-neutral secure-storage abstraction.
• **iOS follow-on:** add the iOS platform/redirect configuration and Keychain-backed implementation when Phase 4 begins; keep the same Entra app registration and application-layer interfaces.
• Prefer delegated access because this is a user companion app acting on behalf of the signed-in user.

10.2 Microsoft Graph permissions

Scope	MVP	Rationale
openid / profile	Yes	Sign-in and identity claims.
offline_access	Yes	Refresh delegated access without repeated interactive sign-in, subject to tenant policy.
Files.ReadWrite.AppFolder	Yes*	Preferred least-privilege data-plane scope; validate support/status for the target tenant before coding.
Files.ReadWrite	No by default	Only if an explicit user-selected-file feature requires it.
Mail/Calendar/Teams scopes	No by default	Core bridge transport does not need broad M365 access.

10.3 Desktop bridge authentication

• **Reuse the same Scout Remote Entra application registration** used by the Android client. Configure the required desktop/native redirect platform(s) on that public-client registration.
• The local MCP bridge/helper signs the user in independently with MSAL and requests the same delegated Graph scope. It does **not** receive, import, copy, or refresh the mobile app's tokens.
• Require the desktop bridge account to match the mobile-bound Microsoft 365 identity. Because both clients call Graph with the same app ID and user, /me/drive/special/approot resolves to the same application folder.
• Persist the desktop MSAL cache in OS-protected storage. Try silent acquisition first; require interactive sign-in again when tenant policy or token state requires it.
• When iOS is added, it authenticates independently under the same app registration and never shares Android or desktop tokens.

Release-scope rule

Android and desktop identity configuration are launch requirements. iOS redirect registration and Keychain implementation are explicitly deferred to Phase 4; do not let those tasks block the Android MVP.

11. Security and threat model

Threat	Control
Stolen phone	OS device lock; secure token cache; optional biometric re-auth for sensitive views; remote account revocation.
Malicious/stale queued command	User identity binding, created/expiry timestamps, strict schema, allowed command types, idempotency receipts.
Privilege expansion	App Folder scope by default; no broad Graph scopes unless feature-specific.
Replay / duplicate processing	Unique commandId + processed receipt; bridge must never re-execute a completed command.
Prompt injection through protocol data	Treat all text fields as untrusted input; skill separates data from instructions and enforces command policy.
Secrets leaked to OneDrive	Protocol schema forbids API keys, refresh tokens, passwords, device codes, connection strings, or raw auth headers.
Remote approval bypass	No mobile approval capability in MVP; surface “open Scout desktop to approve.”
User edits protocol files manually	Validate schema/version; detect malformed state; preserve an audit event; never trust user-editable fields as authorization.

11.1 Data classification

- Status and activity: potentially confidential because task names may reveal work context.
- Conversation text: potentially sensitive; allow configurable retention and “summary only” mode.
- Artifacts: keep as OneDrive items and reference by ID rather than duplicating large content into protocol JSON.
- Telemetry: store counts and model/tool identifiers only when available; never store access tokens or full prompts just for cost accounting.

12. Scout Remote Agent Plugin

Package the desktop bridge as an Agent Plugins 1.0 bundle so the reusable Scout skill and its local MCP bridge travel together. Agent Plugins 1.0 standardizes packaging for Agent Skills and MCP servers; it does not by itself make Scout a conformant plugin installer.

Packaging rule

Keep the plugin as the portable source-of-truth. For current Scout deployments, install/discover the contained SKILL.md through Scout's supported skills directory and register the contained MCP server through Scout's supported MCP configuration path. If Scout later documents native Agent Plugins installation, the same package becomes directly installable without changing the bridge contract.

Conceptual package layout

```
scout-remote-plugin/
  plugin.json
  skills/
    scout-remote-bridge/
      SKILL.md
  mcp.json
  bridge/
    ScoutRemote.Bridge
```

Plugin component	Responsibility
SKILL.md	Agent-facing instructions: when to poll, how to interpret validated bridge state, response formatting, and when to stop/ask for desktop attention.
Local MCP bridge/helper	Owns desktop MSAL sign-in, Graph App Folder I/O, schema/version checks, protocol serialization, and token handling. No secrets returned to Scout.
mcp.json	Declares the local bridge server for clients that implement Agent Plugins/MCP configuration.
plugin.json	Declares Agent Plugins 1.0 package identity and portable discovery layout.

12.1 Bridge prompt-cost policy

Request class	Preferred path	Work IQ posture
Status / bridge history	Read bridge state/activity through the local MCP helper.	Never needed.
OneDrive artifact lookup	Direct Graph through the bridge helper or supported Scout file tool.	Avoid.
Simple mail/calendar fetch	Direct M365 tool when deterministic.	Avoid unless semantic synthesis adds value.
Cross-M365 “what changed?”	Use Work IQ when it materially reduces orchestration.	Allowed within user guardrail.
Dynamics 365 CRUD/query	Direct supported D365 Web API/tool when configured.	Do not use merely for transport.

13. Usage and cost controls

The app should help tune cost without pretending it knows pricing that the underlying services do not expose.

Control	Behavior
Work IQ budget per command	Optional maxWorkIqCalls integer; bridge reports if it cannot satisfy within the bound.
Prompt mode	Efficient / Balanced / Deep. Efficient prefers deterministic tools and concise responses.
Usage event	Record tool name, call count, model identifier if exposed, token counts if exposed, and user-defined estimated unit cost.
Daily summary	Aggregate commands, Work IQ calls, model tokens, failures, and estimated spend.
Cost disclaimer	Estimated spend is user-configured unless an authoritative metering API is available.

Current user preference captured in the design

Treat reasonable Work IQ consumption as acceptable when it adds real value, but actively minimize unnecessary calls. Do not route Dynamics 365 access through Work IQ merely for connectivity. Allow the user to configure external model price assumptions (for example, a custom Grok API rate) rather than hard-coding vendor pricing.

telemetry/usage-daily.json - illustrative, not vendor billing data

```
{
  "date": "2026-08-13",
  "commands": 14,
  "workIqCalls": 3,
  "modelUsage": [
    {
      "provider": "custom",
      "model": "grok-4.6",
      "inputTokens": 82000,
      "outputTokens": 12000,
      "estimatedUsdPerMillionTokens": 2.00,
      "pricingSource": "user-configured"
    }
  ],
  "estimatedTotalUsd": 0.19
}
```

14. Optional Dynamics 365 direct integration

Dynamics 365 is outside the core bridge. Add it only if the mobile experience needs direct D365 data rather than asking Scout to perform a task.

Decision	Recommendation
Authentication	Use a dedicated Entra app registration and supported delegated OAuth/MSAL flow.
Token storage	Secure mobile token cache only; never OneDrive protocol files.
API surface	Use the supported D365 Web API appropriate to the target application/environment.
Authorization	Respect environment security roles and least privilege; do not emulate another user.
Work IQ	Optional for semantic synthesis, not required for basic D365 connectivity.
Device-code prototype	Permissible as a temporary engineering workflow where appropriate; not the production mobile credential model.

Separation of concerns

Keep D365 credentials and API logic in a D365 adapter. The Scout Remote bridge contract should refer to logical business requests/results, not expose or transport D365 access tokens.

15. Reliability, offline behavior, and error handling

Condition	Required behavior
Scout desktop offline	Mobile accepts command into queue and displays “queued - Scout last seen …”.
OneDrive/Graph unavailable	Cache unsent draft locally; retry with exponential backoff; never duplicate commandId.
Command expires before pickup	Bridge writes expired disposition; mobile offers “resend as new command.”
Schema version mismatch	Stop processing incompatible major version; surface upgrade required.
Partial write / corrupt JSON	Ignore incomplete item, log protocol error, never execute ambiguous instruction.
Desktop approval needed	Return waiting_for_desktop_approval with exact reason; do not loop or spam.
Work IQ budget exceeded	Return partial/blocked response and ask whether to retry with a higher budget.
Artifact deleted/moved	Resolve by driveItemId when possible; mark unavailable if no longer accessible.

15.1 Freshness rules

- Every status card displays lastSeenUtc; never show “online” without freshness evidence.
- Default stale threshold should be configurable because heartbeat may run every 15-120 minutes.
- A completed command response is immutable; later corrections are new events referencing the original command.

16. Mobile implementation architecture

Keep the mobile app layered so the transport can change later if Microsoft releases a native Scout API, and so Android-first delivery does not turn into Android-only architecture.

Layer	Responsibilities	Example types
Presentation	Views, navigation, loading/error/empty states, accessibility.	HomeViewModel, ChatViewModel, UsageViewModel
Domain	Agent state, commands, responses, activity, artifacts, cost policy.	AgentStatus, RemoteCommand, RemoteResponse
Application services	Sync, send command, refresh status, reconcile activity.	BridgeSyncService, CommandService
Transport adapter	Graph App Folder implementation; future Scout API adapter.	GraphBridgeTransport
Identity	MSAL sign-in, scopes, account switch, token acquisition.	IdentityService
Persistence	Encrypted local cache, drafts, last sync token, UI preferences.	SQLite / secure storage
Platform services	Secure storage, background work, notifications, auth callback/browser, external app launching.	ISecureStorage, IBackgroundScheduler, INotificationService

16.1 Android-first implementation rule

• Implement and production-test Android platform services first: Keystore-backed secure storage, Android background scheduling constraints, notification behavior, MSAL redirect handling, and Google Play signing/package workflow.
• Keep shared domain, application-service, transport, and protocol code free of Android-specific APIs. Platform-specific code should sit behind interfaces that can receive iOS implementations later.
• Do not stub iOS into the Android release path. The iOS implementation is a later compatibility target against the same interface and contract tests.

16.2 Sync strategy

• Foreground: refresh protocol, status, active conversation, then activity delta.
• Background: lightweight status/response checks at OS-permitted intervals; push can be added later via a backend subscription service if justified.
• Use ETags/If-Match for mutable files such as status/settings; use immutable IDs for commands/responses/events.
• Store the Graph item IDs and delta cursors needed to avoid repeatedly listing the entire app folder.

17. Deployment modes

Mode	Architecture	Best for	Trade-off
A. Personal / no backend	Mobile + Graph App Folder + Scout bridge.	Fast MVP, one user.	Notifications depend on refresh/background fetch.
B. Notification service	Add minimal Azure Function/API + Graph subscription/webhook + APNs/FCM.	Nearer-real-time completion alerts.	Operational cost and app-service security.
C. Enterprise service	Backend brokers policy, audit, multi-user mapping, push, admin controls.	Managed organizational deployment.	Much larger security/compliance scope.

MVP recommendation

Start with Mode A. Prove the bridge protocol and mobile UX before introducing a cloud backend. Add Mode B only if notification latency becomes a real usability problem.

18. Test strategy and acceptance criteria

18.1 Android MVP functional acceptance

• Android user can sign in with Entra ID and grant only the scopes required for the MVP.
• Android app creates/locates the OneDrive App Folder and validates protocol.json.
• Home screen displays explicit freshness and agent state; a mobile message produces one command file with a unique ID and expiry.
• Bridge processes the command once, publishes response, and records processed disposition; the app displays the response and artifact references after refresh.
• Offline Scout leaves the command queued without data loss or false completion; expired/replayed/malformed commands do not execute.
• Sensitive work returns a desktop-attention state rather than a remote bypass.
• Android secure storage, auth callback handling, background behavior, notifications, and offline drafts pass device-level tests.
• A signed Android release build is ready for Google Play internal/closed testing. iOS is not a Phase 1 acceptance requirement.

18.2 Security tests

Test	Pass condition
Scope audit	No broader Graph scope is present than the enabled feature requires.
Token inspection	No access/refresh/device token appears in logs, protocol folder, crash report, or analytics. Android tokens remain in Keystore-backed storage.
Replay	Re-uploading a processed command does not execute it again.
Prompt injection	Malicious text in a command cannot alter bridge policy or permission rules.
Stale command	Expired command is rejected deterministically.
Lost device	Revoking account/session prevents further Graph sync.

Future iOS acceptance

Phase 4 repeats the platform-specific acceptance suite for iOS: MSAL redirect/platform configuration, Keychain storage, background execution behavior, notifications, App Store signing/package flow, and regression against the same domain/transport contract tests.

19. Delivery roadmap

Phase	Scope	Exit criteria
0 - Android spike	Android + desktop MSAL; Agent Plugin package; Graph App Folder; hand-authored protocol files; manual Scout invocation.	One Android-to-desktop round trip works; Android and desktop independently reach the same App Folder without token sharing.
1 - Android MVP	Home, Activity, bridge Chats, Artifacts, Settings; packaged skill + MCP bridge; Android secure storage/background behavior; Google Play packaging; no backend.	Reliable single-user Android daily use; signed build ready for Play internal/closed test; desktop re-auth and offline handling proven.
2 - Cost + polish	Usage screen, Work IQ budget controls, stronger diagnostics, retention controls.	User can tune prompts/cost without inspecting files.
3 - Notifications	Optional Azure notification service and Graph subscriptions.	Completion/attention notifications arrive promptly and securely on Android.
4 - iOS	Add iOS MSAL platform config, Keychain storage, background behavior, notifications, packaging/signing, and platform regression tests.	iOS passes the same domain/transport contract suite and is ready for App Store release workflow.
5 - Adapters	Official Scout API/native-chat adapter if available; optional D365 direct adapter.	Adapters pass contract tests without changing mobile domain model.
6 - Enterprise	Admin policy, app configuration, audit export, managed deployment.	Ready for multi-user organizational governance.

19.1 Android MVP backlog

• Create one Entra public-client app registration with Android and desktop/native redirect configuration; defer iOS platform configuration to Phase 4.
• Build the Graph App Folder client and prove Android + desktop helper resolve the same approot for the same signed-in user.
• Package Scout Remote Bridge as an Agent Plugins 1.0 directory with plugin.json, skills/.../SKILL.md, and mcp.json.
• Implement Android Keystore-backed secure storage, MSAL redirect handling, background scheduling behavior, notifications, encrypted local cache, offline drafts, and idempotent retry behind platform-service interfaces.
• Implement Home + Activity + Chats + Artifacts + Settings and add usage counters.
• Create a signed Android release package suitable for Google Play internal/closed testing.
• Run Android auth/account-binding, device-loss, background/offline, security/replay/expiry tests and a 72-hour reliability soak.

20. Open questions and decision log

Question	Default decision for v1	Revisit when
Can native Scout chats be listed through a supported API?	Assume no; use bridge-owned chats.	Microsoft publishes documented session/chat API.
Can Scout publish event callbacks faster than heartbeat?	Assume asynchronous polling/automation.	Supported webhook/event/agent API exists.
Can mobile approve Scout sensitive actions?	No.	Microsoft exposes explicit secure approval API and policy model.
Should the app have a backend?	No for MVP.	Push latency, multi-user policy, or audit requirements justify it.
Should D365 be integrated?	Optional adapter only.	A concrete mobile D365 scenario requires it.
How should Work IQ cost be shown?	Observed call counts + user-configured estimate.	Authoritative tenant/user metering API is available.

Architecture invariant

Even if Microsoft later adds first-party Scout mobile APIs, preserve the domain-level interfaces (status, activity, remote command, response, artifact, usage). Swap the transport adapter instead of rewriting the app.

21. Operational runbook

21.1 First-time setup - Android MVP

1. Install/configure Scout on the desktop and confirm Microsoft 365 sign-in.

2. Register Scout Remote as one Entra public-client application. Configure Android and desktop/native platform redirects for the MVP; leave iOS platform configuration for Phase 4.

3. Install the Scout Remote Agent Plugin source package. Until Scout documents native Agent Plugins installation, place the contained SKILL.md in a supported Scout skills directory and register the contained local MCP bridge with Scout.

4. Run the desktop bridge sign-in once. Confirm it uses the Scout Remote app ID, signs in as the intended Microsoft 365 user, and stores its MSAL cache only in OS-protected local storage.

5. Create/enable the bridge heartbeat or automation and set an acceptable work-hours/cadence policy.

6. Install the Android Scout Remote build and sign in with the same Microsoft 365 identity. Confirm Android uses Keystore-backed secure storage through the platform abstraction.

7. From Android and desktop, resolve /me/drive/special/approot and verify both clients see the same protocol bootstrap.

8. Run “Request fresh status” from Android and confirm a complete round trip. For release candidates, repeat after app restart, device reboot, Scout-offline recovery, and account re-authentication.

21.2 Troubleshooting order

1. Check Android sign-in, Graph consent, bound Entra account, MSAL redirect handling, and secure-storage access.

2. Check desktop bridge MSAL sign-in and confirm the same app ID + same M365 user are in use; never copy tokens between devices.

3. Verify Android and desktop resolve the same App Folder and protocol.json is readable.

4. Confirm the local MCP bridge is registered/allowed by Scout policy and its process can start.

5. Check lastSeenUtc and automation/heartbeat history on Scout desktop.

6. Inspect inbox command JSON for schema/expiry issues, then the processed receipt/response.

7. Only then inspect Scout model/tool execution details.

Phase 4 iOS setup delta

Add the iOS Entra platform/redirect configuration, Keychain-backed secure storage implementation, iOS background/notification behavior, and App Store signing/package workflow. The bridge and protocol setup stay unchanged.

Appendix A. State machines

A.1 Command state machine

```
draft
  -> queued
  -> acknowledged
  -> running
  -> completed
       |-> failed
       |-> expired
       |-> waiting_for_desktop_approval
       |-> canceled   (only when confirmed by bridge)
```

A.2 Agent presence state

```
unknown -> idle -> working -> idle
                |          |
                |          +-> waiting_for_desktop
                +-> stale/offline (derived from lastSeenUtc)

Presence is evidence-based. The mobile client derives stale/offline from freshness;
Scout does not need to maintain a separate always-online socket.
```

Appendix B. Retention defaults

Data	Suggested default	User control
Status	Latest only	None needed beyond reset.
Activity	30 days	7 / 30 / 90 days / manual.
Remote conversation events	90 days	Per-thread delete/export; summary-only mode.
Processed receipts	30 days or command retention period	Automatic cleanup.
Artifacts	Do not duplicate; retain references	Artifact follows OneDrive lifecycle.
Usage aggregates	90 days	30 / 90 / 365 days / off.

Appendix C. Reference basis

These public sources underpin the supported surfaces used by the design. URLs are included in the source document for engineering verification because Scout is rapidly evolving and Agent Plugins 1.0 is new.

Source	Used for
Microsoft Scout overview / common questions	Desktop platforms, custom skills, heartbeat/automations, M365 connection, current mobile limitation.
Microsoft Scout admin controls	Evidence that Scout exposes MCP-server policy/controls; enterprise policy may block MCP, heartbeat, or automations.
Agent Plugins Specification 1.0.0 (agent-plugins.org)	Portable plugin package format; Agent Skills in skills/ and MCP server configuration in mcp.json.
VS Code Agent Plugins documentation	Cross-tool implementation example and compatibility model; client-specific capabilities remain outside portable core.
Microsoft Graph - Using app folder in OneDrive and SharePoint	App Folder access model and least-privilege cross-device data plane.
Microsoft Graph / OneDrive permissions references	Current Files.ReadWrite.AppFolder permission status and caveats; must be revalidated for target tenant.
Microsoft identity platform - desktop app configuration	Desktop public-client application configuration, native redirect patterns, and interactive MSAL sign-in.
Microsoft identity platform - mobile authentication / MSAL flows	Mobile public-client authentication, PKCE, and token-cache guidance.
Agent Plugins announcement / governance sources	Confirms vendor-neutral launch and that the 1.0 portable component types are Agent Skills and MCP servers.

Validation checkpoint before coding

Re-check Scout documentation for native Agent Plugins installation and MCP registration; re-check Microsoft Graph App Folder permission support/status in the target Microsoft 365 tenant; and re-check any newly published Scout developer APIs. If Scout adds first-party mobile/session APIs, implement them behind the existing transport adapter. If Scout gains native Agent Plugins support, install the same package directly instead of changing the bridge design.